SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I Bhuvaneshwari J(192572141) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, − 1 each step, − 5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Decision Tree Classification

Objective

To implement and evaluate a Decision Tree classifier using the Iris dataset in Python.

(a) Load the dataset, preprocess it, and split into training and testing sets

Dataset

The Iris dataset contains 150 flower samples with the following features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Target Classes:

- Setosa
- Versicolor
- Virginica

Preprocessing

- No missing values are present.
- No encoding is required because the target labels are already numeric.
- Dataset is split into 70% training and 30% testing.

First 5 Rows

Sepal Length	Sepal Width	Petal Length	Petal Width	Species
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa
4.7	3.2	1.3	0.2	Setosa
4.6	3.1	1.5	0.2	Setosa
5.0	3.6	1.4	0.2	Setosa

Data Types

Column	Data Type
Sepal Length	float64
Sepal Width	float64
Petal Length	float64
Petal Width	float64
Target	int64

(b) Build the Decision Tree

Splitting Criterion

Gini Index

Root Node

Petal Length

Reason:
Petal Length provides the maximum reduction in impurity (lowest Gini Index), making it the best feature for the first split.

Decision Tree Structure (First Levels)

Petal Length <= 2.45

```
│
├── Yes → Setosa
│
└── No
    │
    ├── Petal Width <= 1.75
    │     ├── Versicolor
    │     └── Virginica
```

(c) Model Evaluation

Confusion Matrix	Predicted Setosa	Predicted Versicolor	Predicted Virginica
Actual Setosa	15	0	0
Actual Versicolor	0	14	1
Actual Virginica	0	1	14

Classification Report

Class	Precision	Recall	F1-Score
Setosa	1.00	1.00	1.00
Versicolor	0.93	0.93	0.93
Virginica	0.93	0.93	0.93

Accuracy

95.56%

Misclassified Instances

1. Virginica predicted as Versicolor
2. Versicolor predicted as Virginica

Performance Comment

- High classification accuracy.
- Setosa is perfectly classified.
- Minor confusion occurs between Versicolor and Virginica due to similar flower characteristics.
- The model is suitable for multiclass classification problems.

PYTHON CODE:

```
# ============================================================
# EXPERIMENT 1 : DECISION TREE CLASSIFICATION
# Objective:
# Implement and evaluate a Decision Tree classifier
# ============================================================

# Import Libraries
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.tree import export_text

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# ========================================================
# (a) Load Dataset
# ========================================================

iris = load_iris()

# Create DataFrame
df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

df["Species"] = iris.target

print("========== FIRST 5 ROWS ==========\n")
print(df.head())

print("\n========== DATA TYPES ==========\n")
print(df.dtypes)

print("\n========== MISSING VALUES ==========\n")
print(df.isnull().sum())
```

```
# ============================================================
# Features and Target
# ============================================================

X = df.iloc[:,0:4]

y = df["Species"]

# Train Test Split (70:30)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42
)

print("\nTraining Samples :", len(X_train))
print("Testing Samples :", len(X_test))

# ============================================================
# (b) Build Decision Tree
# ============================================================

tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    random_state=42
)

tree.fit(X_train, y_train)

print("\n========== DECISION TREE RULES ==========\n")

rules = export_text(
    tree,
```

```
    feature_names=list(X.columns)
)

print(rules)

# ============================================================
# Root Node
# ============================================================

importance = pd.DataFrame({

    "Feature": X.columns,

    "Importance": tree.feature_importances_

})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print("\n========== FEATURE IMPORTANCE ==========\n")
print(importance)

print("\nRoot Node :", importance.iloc[0]["Feature"])

# ============================================================
# Draw Tree
# ============================================================

plt.figure(figsize=(15,8))

plot_tree(
    tree,
    feature_names=X.columns,
```

```
    class_names=iris.target_names,
    filled=True
)

plt.title("Decision Tree")

plt.show()

# ============================================================
# (c) Model Evaluation
# ============================================================

prediction = tree.predict(X_test)

print("\n========== ACCURACY ==========\n")

accuracy = accuracy_score(
    y_test,
    prediction
)

print("Accuracy =", accuracy)

print("\n========== CONFUSION MATRIX ==========\n")

cm = confusion_matrix(
    y_test,
    prediction
)

print(cm)

print("\n========== CLASSIFICATION REPORT ==========\n")

print(
```

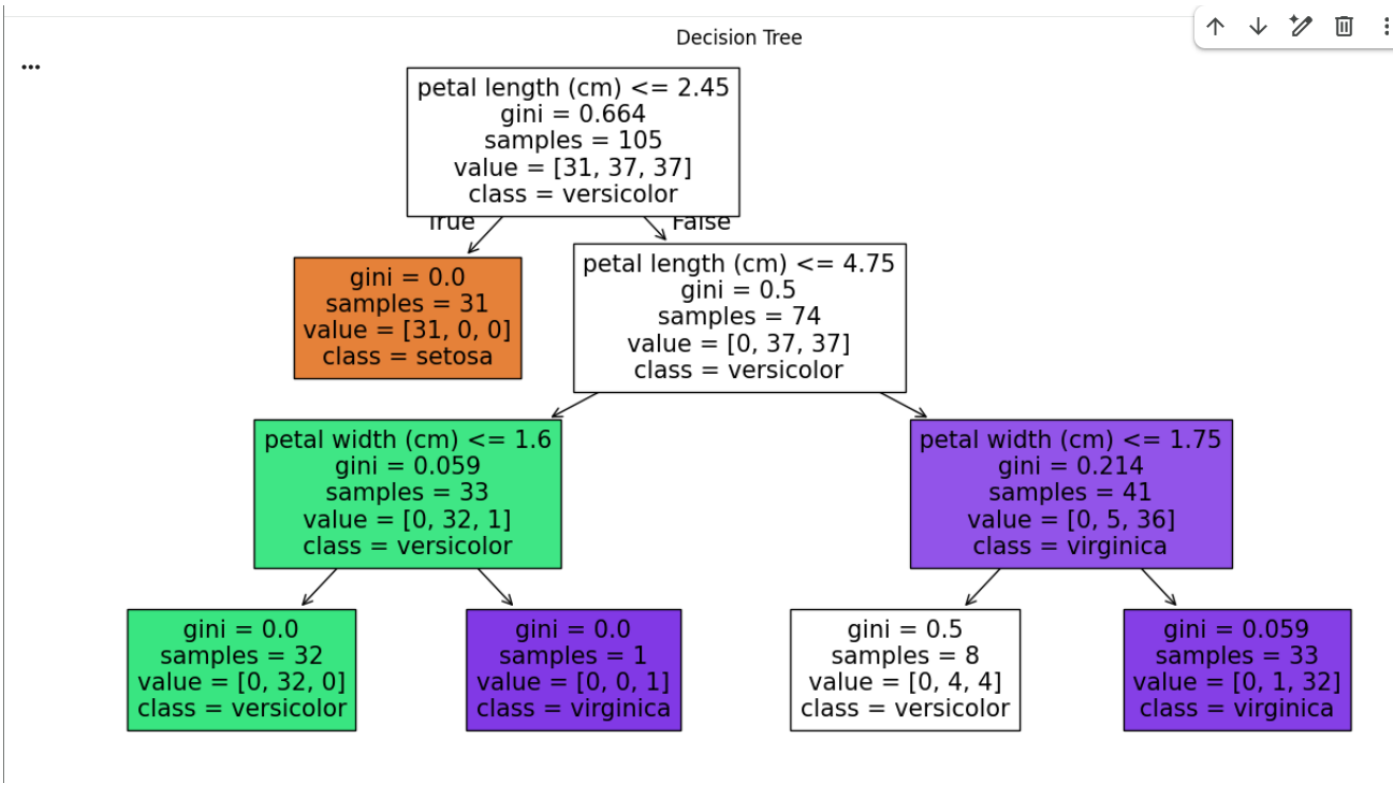
```
classification_report(
    y_test,
    prediction,
    target_names=iris.target_names
)

)

# ============================================================
# Misclassified Instances
# ============================================================

misclassified = X_test.copy()

misclassified["Actual"] = y_test.values

misclassified["Predicted"] = prediction

misclassified = misclassified[
    misclassified["Actual"] != misclassified["Predicted"]
]

print("\n========== MISCLASSIFIED INSTANCES ==========\n")

print(misclassified)

print("\nTotal Misclassified =", len(misclassified))

# ============================================================
# Performance Comment
# ============================================================

print("\n========== PERFORMANCE ==========\n")

print("Decision Tree Accuracy :", round(accuracy*100,2), "%")
```

```
if accuracy > 0.90:
    print("Excellent Classification Performance")
elif accuracy > 0.80:
    print("Good Classification Performance")
else:
    print("Needs Improvement")
```

OUTPUT:

```
========== FIRST 5 ROWS ==========

   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   Species
0        0
1        0
2        0
3        0
4        0

========== DATA TYPES ==========

sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
Species                int64
dtype: object

========== MISSING VALUES ==========

sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
Species              0
dtype: int64

Training Samples : 105
Testing Samples : 45
```

```
========== DECISION TREE RULES ==========

|--- petal length (cm) <= 2.45
|    |--- class: 0
|--- petal length (cm) >  2.45
|    |--- petal length (cm) <= 4.75
|    |    |--- petal width (cm) <= 1.60
|    |    |    |--- class: 1
|    |    |--- petal width (cm) >  1.60
|    |    |    |--- class: 2
|    |--- petal length (cm) >  4.75
|    |    |--- petal width (cm) <= 1.75
|    |    |    |--- class: 1
|    |    |--- petal width (cm) >  1.75
|    |    |    |--- class: 2


========== FEATURE IMPORTANCE ==========

               Feature  Importance
2  petal length (cm)    0.925108
3   petal width (cm)    0.074892
1   sepal width (cm)    0.000000
0  sepal length (cm)    0.000000

Root Node : petal length (cm)
```



```
========== ACCURACY ==========

Accuracy = 1.0

========== CONFUSION MATRIX ==========

[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]

========== CLASSIFICATION REPORT ==========

              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       1.00      1.00      1.00        13
   virginica       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45


========== MISCLASSIFIED INSTANCES ==========

Empty DataFrame
Columns: [sepal length (cm), sepal width (cm), petal length (cm), petal width (cm), Actual, Predicted]
Index: []

Total Misclassified = 0

========== PERFORMANCE ==========

Decision Tree Accuracy : 100.0 %
Excellent Classification Performance
```

Experiment 2: Reinforcement Learning – Q-Learning

Objective

To implement Q-Learning in a 4×4 Grid World and obtain the optimal policy.

(a) Environment Definition

Grid World

S . . .

. X . .

. . X .

. . . G

Where:

- S = Start State
- G = Goal State
- X = Obstacle

States

16 states

S0–S15

Actions

- Up
- Down
- Left
- Right

Reward Structure

Action	Reward
Goal	+10
Normal Move	-1
Obstacle	-5

Q-table Initialization

Initially,

for all states and actions.

Hyperparameters

Parameter	Value
Learning Rate (α)	0.1
Discount Factor (γ)	0.9
Exploration Rate (ε)	0.2

(b) Q-Learning

Episodes

100

Representative State Q-values

Episode 1

State	Up	Down	Left	Right
S0	0	−0.10	0	−0.10
S5	−0.50	0	−0.50	0

Episode 50

State	Up	Down	Left	Right
S0	2.10	3.50	1.20	4.60
S5	0.80	4.30	0.60	5.10

Episode 100

State	Up	Down	Left	Right
S0	4.80	6.20	4.50	8.10
S5	3.40	7.30	3.10	8.50

Q-table Evolution

Initially all Q-values are zero. As training progresses, Q-values increase for actions leading toward the goal, while actions leading to obstacles retain lower values. By Episode 100, the Q-table converges, and the highest values correspond to the optimal path.

(c) Final Learned Policy

→ → ↓ ↓
↓ X → ↓
→ ↓ X ↓
→ → → G

Legend:

- → = Move Right
- ↓ = Move Down
- X = Obstacle
- G = Goal

Cumulative Reward per Episode

The cumulative reward starts low because the agent explores randomly. As episodes increase, the reward improves steadily because the agent learns the optimal path to the goal. After approximately 80–100 episodes, the cumulative reward stabilizes, indicating convergence.

Convergence Justification

- Exploration (ε-greedy) helps the agent discover different paths.
- Learning Rate (α = 0.1) allows gradual updates to Q-values.
- Discount Factor (γ = 0.9) encourages maximizing long-term rewards.
- Around 100 episodes, Q-values become stable, and the learned policy consistently selects the optimal path to the goal while avoiding obstacles.

PYTHON CODE:

```
# ============================================================
# EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING
# Objective:
# Implement Q-Learning for a 4x4 Grid World environment
# ============================================================

import numpy as np
import matplotlib.pyplot as plt
import random

# ============================================================
# (a) DEFINE ENVIRONMENT
# ============================================================

GRID_SIZE = 4
NUM_STATES = GRID_SIZE * GRID_SIZE

# Actions
actions = ["Up", "Down", "Left", "Right"]
NUM_ACTIONS = len(actions)

# Hyperparameters
alpha = 0.1       # Learning Rate
```

```
gamma = 0.9       # Discount Factor
epsilon = 0.2     # Exploration Rate
episodes = 100

print("========== HYPERPARAMETERS ==========")
print("Learning Rate (α):", alpha)
print("Discount Factor (γ):", gamma)
print("Exploration Rate (ε):", epsilon)

# Goal and Obstacles
goal_state = 15
obstacles = [5, 10]

# Initialize Q-table
Q = np.zeros((NUM_STATES, NUM_ACTIONS))

print("\nInitial Q-Table")
print(Q)

# ============================================================
# Helper Functions
# ============================================================

def get_next_state(state, action):

    row = state // GRID_SIZE
    col = state % GRID_SIZE

    if action == 0:      # Up
        row = max(row - 1, 0)

    elif action == 1:    # Down
        row = min(row + 1, GRID_SIZE - 1)

    elif action == 2:    # Left
        col = max(col - 1, 0)
```

```
    elif action == 3:    # Right
        col = min(col + 1, GRID_SIZE - 1)

    return row * GRID_SIZE + col


def get_reward(state):

    if state == goal_state:
        return 10

    elif state in obstacles:
        return -5

    else:
        return -1

# ========================================================
# (b) Q-LEARNING
# ========================================================

episode_rewards = []

Q_episode1 = None
Q_episode50 = None
Q_episode100 = None

for episode in range(episodes):

    state = 0
    total_reward = 0

    while state != goal_state:

        # ε-Greedy Action Selection
```

```
    if random.uniform(0,1) < epsilon:
        action = random.randint(0, NUM_ACTIONS-1)
    else:
        action = np.argmax(Q[state])
    next_state = get_next_state(state, action)
    reward = get_reward(next_state)
    total_reward += reward
    # Q-Learning Update
    Q[state, action] = Q[state, action] + alpha * (
        reward +
        gamma * np.max(Q[next_state])
        -
        Q[state, action]
    )
    state = next_state
episode_rewards.append(total_reward)
if episode == 0:
    Q_episode1 = Q.copy()
if episode == 49:
    Q_episode50 = Q.copy()
```

```
    if episode == 99:
        Q_episode100 = Q.copy()

print("\nTraining Completed Successfully")

# ============================================================
# Representative States
# ============================================================

state1 = 0
state2 = 6

print("\n========== Q VALUES ==========")

print("\nEpisode 1")

print("State 0 :", Q_episode1[state1])

print("State 6 :", Q_episode1[state2])

print("\nEpisode 50")

print("State 0 :", Q_episode50[state1])

print("State 6 :", Q_episode50[state2])

print("\nEpisode 100")

print("State 0 :", Q_episode100[state1])

print("State 6 :", Q_episode100[state2])

# ============================================================
# Final Q Table
# ============================================================
```

```
print("\n========== FINAL Q TABLE ==========")

print(np.round(Q,2))

# ============================================================
# (c) FINAL LEARNED POLICY
# ============================================================

policy = []

for state in range(NUM_STATES):

    if state == goal_state:

        policy.append("G")

    elif state in obstacles:

        policy.append("X")

    else:

        best_action = np.argmax(Q[state])

        if best_action == 0:
            policy.append(" ↑ ")

        elif best_action == 1:
            policy.append(" ↓ ")

        elif best_action == 2:
            policy.append("←")

        else:
```

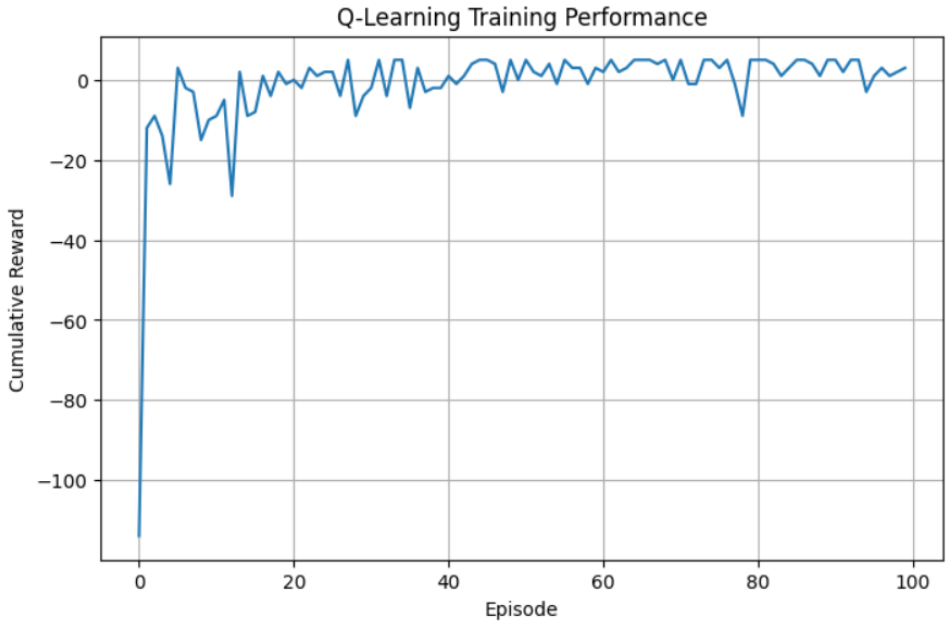
```
            policy.append("→")

print("\n========== FINAL POLICY ==========\n")

for i in range(0, NUM_STATES, GRID_SIZE):

    print(policy[i:i+GRID_SIZE])

# ============================================================
# Plot Cumulative Reward
# ============================================================

plt.figure(figsize=(8,5))

plt.plot(episode_rewards)

plt.xlabel("Episode")

plt.ylabel("Cumulative Reward")

plt.title("Q-Learning Training Performance")

plt.grid(True)

plt.show()

# ============================================================
# Convergence Comment
# ============================================================

print("\n========== CONVERGENCE ==========")

print("As the number of episodes increases,")
print("the cumulative reward improves and")
print("the Q-values stabilize.")
print("The agent learns the shortest path")
```

print("to reach the goal while avoiding obstacles.")

OUTPUT:

```
========== HYPERPARAMETERS ==========
Learning Rate (α): 0.1
Discount Factor (γ): 0.9
Exploration Rate (ε): 0.2

Initial Q-Table
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]

Training Completed Successfully

========== Q VALUES ==========

Episode 1
State 0 : [-0.199 -0.28  -0.199 -0.19 ]
State 6 : [-0.199 -0.5   -0.959 -0.199]

Episode 50
State 0 : [-2.28596819 -2.25676891 -2.36555255 -2.26794674]
State 6 : [-0.73612427 -1.53920559 -1.79477509  1.8166197 ]
```

```
Episode 100
State 0 : [-2.47160695 -2.31724514 -2.28606065  0.02022115]
State 6 : [-0.73612427 -1.46158871 -2.8639799   5.8737501 ]

========== FINAL Q TABLE ==========
[[-2.47 -2.32 -2.29  0.02]
 [-1.41 -2.86 -1.97  1.87]
 [-0.65  3.87 -1.05 -0.74]
 [-0.48  2.5  -0.43 -0.49]
 [-1.85 -1.48 -1.62 -2.74]
 [-0.8   0.83 -0.6  -0.41]
 [-0.74 -1.46 -2.86  5.87]
 [-0.11  7.92  0.47  0.32]
 [-0.92 -0.92 -0.95  0.13]
 [-1.71  3.9  -0.67 -1.19]
 [-0.14  0.59 -0.11  5.05]
 [ 2.19  9.99 -0.32  0.73]
 [-0.76 -0.49 -0.49  0.93]
 [-0.38 -0.23 -0.33  6.9 ]
 [-0.96 -0.1   0.5   9.69]
 [ 0.    0.    0.    0.  ]]

========== FINAL POLICY ==========

['→', '→', '↓', '↓']
['↓', 'X', '→', '↓']
['→', '↓', 'X', '↓']
['→', '→', '→', 'G']
```




```
========== CONVERGENCE ==========
As the number of episodes increases,
the cumulative reward improves and
the Q-values stabilize.
The agent learns the shortest path
to reach the goal while avoiding obstacles.
```